



PONTIFICIA UNIVERSIDAD CATOLICA DE CHILE
ESCUELA DE INGENIERIA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACION

Criptografía y Seguridad Computacional - IIC3253

Tarea 3

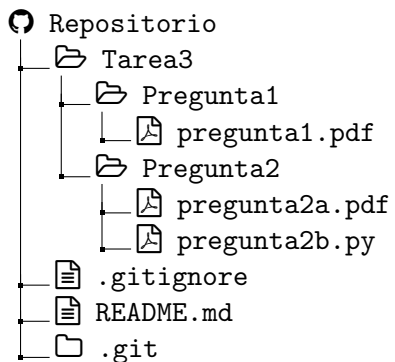
Plazo de entrega: lunes 8 de junio

Instrucciones

Cualquier duda sobre la tarea se deberá hacer en los *issues* del repositorio del curso. Los issues son el canal de comunicación oficial para todas las tareas.

Configuración inicial. Para esta tarea utilizaremos *github classroom*. Para acceder a su repositorio privado debe ingresar a un link que le enviaremos antes de la entrega de la tarea, seleccionar su nombre y aceptar la invitación. El repositorio se creará automáticamente una vez que haga esto y lo encontrará junto a los repositorios del curso. Para la corrección se utilizará Python 3.13.7.

Entrega. Al entregar esta tarea, su repositorio se deberá ver exactamente de la siguiente forma:



Para cada problema cuya solución se deba entregar como un documento, deberá entregar un archivo `.pdf` que, o bien fue construido utilizando $\text{\LaTeX}$, o bien es el resultado de digitalizar un documento escrito a mano. En caso de optar por esta última opción, queda bajo su responsabilidad la legibilidad del documento. Respuestas que no puedan interpretar de forma razonable los ayudantes y profesores, ya sea por la caligrafía o la calidad de la digitalización, serán evaluadas con la nota mínima.

Preguntas

1. En esta pregunta deberá demostrar la correctitud de la formula de Garner, la cual se utiliza para decriptar mensajes usando RSA en OpenSSH. Recuerde que usamos la notación $a^{-1} \bmod b$ para referirnos al inverso de a en módulo b .

Sean (e, N) y (d, N) un par de llaves RSA pública y privada (respectivamente), con $N = P \cdot Q$.

- (a) [2 puntos] Demuestre que para cada par de números $x_1 \in \{0, \dots, P-1\}$ y $x_2 \in \{0, \dots, Q-1\}$, si existe un número $x \in \{0, \dots, N-1\}$ tal que $x \equiv_P x_1$ y $x \equiv_Q x_2$, dicho número es el único que cumple tal condición. Esto es un caso particular del teorema chino de los restos. No utilice dicho teorema en la demostración.
- (b) [2 puntos] Demuestre que para cada texto cifrado $c \in \{1, \dots, N\}$, su correspondiente texto plano se puede calcular como

$$m = (c^d \cdot P \cdot (P^{-1} \bmod Q) + c^d \cdot Q \cdot (Q^{-1} \bmod P)) \bmod N$$

- (c) [2 puntos] Demuestre que para cada mensaje cifrado $c \in \{1, \dots, N\}$, su correspondiente texto plano se puede calcular como

$$m = c^d \bmod Q + (c^d \bmod P - c^d \bmod Q) \cdot (Q^{-1} \bmod P) \cdot Q$$

Esta fórmula se conoce como la fórmula de Garner, y es lo que se utiliza en la práctica para firmar y decriptar mensajes usando RSA.

2. En esta pregunta usted deberá mostrar la correctitud del protocolo RSA entre múltiples partes y luego programar una clase que represente a un participante de dicho protocolo. El objetivo de este protocolo es permitir a un grupo de participantes comunicarse de tal forma que todo participante del grupo puede encriptar mensajes que puede decriptar cualquier participante del grupo.

El protocolo RSA entre múltiples partes funciona de la siguiente forma: Supongamos que tenemos ℓ participantes. Cada participante $i \in \{1, \dots, \ell\}$ genera dos primos grandes P_i y Q_i al azar y comparte su *módulo* $N_i = P_i \cdot Q_i$. Luego el grupo verifica que todos los N_i sean primos relativos; de lo contrario el protocolo comienza de nuevo. Finalmente, el grupo se pone de acuerdo en un *exponente público* $e \in \{1, \dots, \min(N_1, \dots, N_\ell)\}$ (se puede sacar al azar) y verifican que sea primo relativo con $\phi(N_i)$ para cada i . Si no lo es, el protocolo elige un nuevo valor $e \in \{1, \dots, \min(N_1, \dots, N_\ell)\}$ y se repite el procedimiento. Definimos la *llave pública* para encriptar como $(e, N_1, N_2, \dots, N_\ell)$.

Si un participante $i \in \{1, \dots, \ell\}$ quiere encriptar un mensaje $m \in \{0, \dots, \min(N_1, \dots, N_\ell)-1\}$, primero debe calcular, para cada $j \in \{1, \dots, \ell\}$, el valor N_{-j} que es la multiplicación de todos los valores N_k con $k \in \{1, \dots, \ell\} \setminus \{j\}$. Luego, computa $M_j = N_{-j}^{-1} \bmod N_j$. Finalmente, envía el mensaje

$$c = (m^e \bmod N_1) \cdot M_1 \cdot N_{-1} + (m^e \bmod N_2) \cdot M_2 \cdot N_{-2} + \dots + (m^e \bmod N_\ell) \cdot M_\ell \cdot N_{-\ell}$$

- (a) [2 puntos] Explique cómo puede un participante decriptar el mensaje c . Explique cómo se vería su llave secreta y demuestre que el protocolo es correcto (es decir, al decriptar se obtiene el mensaje original). Es importante que su protocolo pueda ser implementado con la interfaz que se describe en la pregunta (b).
- (b) [4 puntos] Escriba una clase en Python que represente a un participante de este protocolo. En particular, su clase deberá respetar al menos la siguiente interfaz.

```
class MultiPartyRSAParticipant:
    def __init__(self, bit_length: int) -> None:
        # Initializes P, Q and N such that N has at least bit_length and at
        # most bit_length + 1 bits

    def get_N(self):
        # Returns N

    def join_group(self, public_exponent: int, group_moduli: List[int]) -
        > boolean:
        # Raises an AssertionError if a group has already been joined.
        # Verifies that N occurs exactly once in group_moduli and that all
        # other moduli are coprime with N. Otherwise, returns False.
        # Verifies that e is coprime with (P - 1)(Q - 1). Otherwise,
        # returns False.
        # Precomputes and stores all relevant information for encryption
        # and decryption, then returns True.

    def enc(self, m: int) -> int:
        # Encrypt m according to the multiparty RSA protocol. If no group
        # has been joined raise an AssertionError.

    def dec(self, c: int) -> int:
        # Decrypt c according to the multiparty RSA protocol. If no group
        # has been joined raise an AssertionError.
```

A continuación se muestra un bloque de código que utiliza la clase anterior, ejemplificando el funcionamiento del protocolo:

```
import random

participant_number = 5
bit_length = 2048

participants = []
for i in range(participant_number):
    participants.append(MultiPartyRSAParticipant(bit_length))

moduli = [p.get_N() for p in participants]
public_exponent = random.randint(1, min(moduli))

for p in participants:
    p.join_group(public_exponent, moduli)
```

```
encryptor = random.choice(participants)

plaintext = random.randint(0, min(moduli) - 1)
cyphertext = encryptor.enc(plaintext)

for p in participants:
    if p.dec(cyphertext) != message:
        raise AssertionError("Incorrect decryption")

print("All good")
```